BRAC University

Department of Computer Science and Engineering

CSE471: System Analysis and Design

Project Report

Project Title: Campus Resource Sharing System

Group No: 01, CSE471	Lab Section: 11, Summer 2025
22301013	Hrithik Dev
22201693	Ramisha Hossain
22301249	Fahim Islam Farhan

Submission Date: [4-5-26]

Table of Contents

1. System Request

Business Need

University students frequently struggle to access essential academic resources such as textbooks, lab equipment, electronics, and course notes. Purchasing these items individually is costly and unsustainable, while informal resource sharing lacks organization and trust. The Campus Resource Sharing System addresses this gap by providing a structured, community-driven platform that enables BRAC University students to lend, borrow, and exchange academic resources efficiently. By digitizing the sharing process with credibility scoring, transaction tracking, and map-based pickup coordination, the system promotes resource sustainability and peer collaboration within the campus community.

Business Requirements

The Campus Resource Sharing System must fulfill the following core requirements:

- Develop a secure and user-friendly web application using Laravel (PHP) and Tailwind CSS for students to post, browse, and manage shared resources.
- Enable real-time borrowing request management with SMS API notifications to resource owners, ensuring timely communication between parties.
- Integrate Google Maps API to display geo-located resource pickup points, helping borrowers easily find resources near their campus area.
- Implement a credibility scoring and review system so users can evaluate transaction partners and build trust within the community.
- Provide automated due-date reminders via push notifications to minimize overdue returns and resource loss.
- Support OCR-powered handwritten PDF-to-text conversion for easy digitization of handwritten notes as shareable resources.
- Maintain an admin panel for oversight, dispute resolution, and penalty enforcement in cases of lost or damaged items.

Business Value

The Campus Resource Sharing System is expected to deliver significant value to the BRAC University student community across multiple dimensions:

- Economic Value: Students can avoid repeated purchases of expensive textbooks and lab equipment by borrowing from peers, reducing individual academic expenditure.
- Social Value: The platform fosters a culture of mutual aid and community trust through credibility scores, peer reviews, and gamified contribution leaderboards with badges.
- Environmental Value: By encouraging resource reuse and reducing redundant purchases, the system contributes to a measurable reduction in resource waste, tracked via environmental impact statistics.
- Operational Value: Automated notifications, structured dashboards, and integrated mapping reduce administrative overhead and improve coordination for both lenders and borrowers.

Over time, the platform is expected to increase student engagement, with top contributors recognized through monthly leaderboards, driving sustainable adoption and long-term community growth.

Special Issues or Constraints

- Trust and Security: As the platform facilitates peer-to-peer exchanges among students, the system must robustly handle credibility scoring to prevent fraudulent lending or borrowing behaviors.
- API Dependencies: The system relies on third-party APIs (Google Maps, SMS, OCR). Downtime or rate limits from these providers may affect core functionality, requiring graceful fallback mechanisms.
- Data Privacy: Student profiles, contact details, and transaction histories must be stored securely in compliance with university data protection policies.
- Deployment Constraints: The system must be deployable on shared hosting or VPS environments (Apache/Nginx), with optional support for Laravel Forge or Railway for CI/CD workflows.
- Scope Limitation: The platform is scoped to BRAC University students only; inter-campus or public access is not a requirement for the current version.

2. Functional Requirements

Module 1: Resource Discovery and Management

1. [Fahim] Users can browse through a categorized list of available resources (textbooks, notes, lab equipment, electronics) posted by other students, with filters for category, condition, and availability status. Recommended resources are shown first based on relevance and credibility of the owner.
2. [Hrithik] Users can post their own resources for sharing by uploading photos, adding descriptions, setting availability duration, and specifying whether the item is for free lending or exchange-based borrowing.
3. [Ramisha] Users can view and edit detailed resource profiles, including the owner's credibility score, past transaction history, and reviews from previous borrowers.

Module 2: Borrowing and Transaction Management

1. [Fahim] Users can send borrow requests to resource owners with proposed pickup dates and return timelines, with automatic notification sent to the owner via SMS API integration.
2. [Ramisha] Users can see their active lending and borrowing transactions on a dashboard showing due dates, overdue items, and pending requests with color-coded status indicators.
3. [Hrithik] Users can locate resource pickup points on an integrated map powered by Google Maps API, showing the exact location of available resources near their campus area.

Module 3: Community Engagement and Reporting

1. [Fahim] Users can rate and review their transaction experience after returning borrowed items, with ratings contributing to the owner's credibility score.
2. [Fahim] Users can convert handwritten PDF documents to editable text using OCR (Google Vision / Tesseract), self-edit the result, and download the converted document.
3. [Ramisha] Users can receive automated reminders via push notifications 24 hours before a borrowed item's due date to avoid late returns.
4. [Hrithik] Users can track their contribution statistics, including total items lent, total resources borrowed, environmental impact (resources saved from purchasing), and community karma points earned.
5. [Ramisha] Users can report lost, damaged, or unreturned resources with evidence upload, triggering an admin review process and a potential penalty for the borrower.
6. [Hrithik] Users can access a leaderboard displaying top contributors of the month based on lending frequency, positive reviews, and community engagement. Badges are awarded to active contributors, and points are added based on reviews received.

3. Technology (Framework, Languages)

Category	Technology / Details
Languages	PHP, JavaScript
Backend Framework	Laravel
Frontend / Templating	Blade Template Engine, Vanilla JavaScript
CSS Framework	Tailwind CSS
Database	MySQL
ORM	Eloquent ORM
Authentication & Authorization	Laravel Authentication, Middleware-based Role Management
API Integrations	Google Maps API, SMS Notification API, OCR API (Google Vision / Tesseract)
File Handling	Laravel File Storage System
Deployment	Shared Hosting / VPS (Apache or Nginx); Optional: Laravel Forge / Railway

4. Backend Development

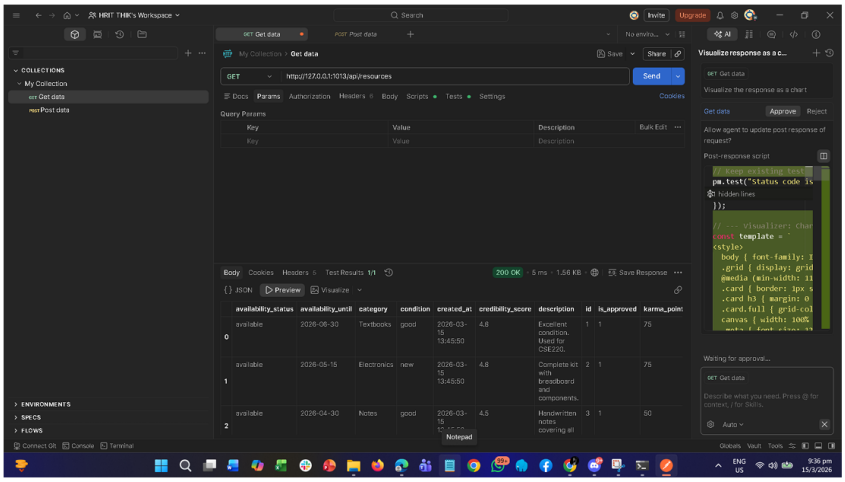
This section presents key backend API implementations with descriptions. Each API has been tested using Postman. The backend is built with Laravel using RESTful conventions and Eloquent ORM for database interaction.

API 1: User Registration (POST /api/register)

This API allows a new student to register on the platform. It validates all required fields, checks for duplicate emails, hashes the password using bcrypt, and stores the new user in the MySQL database. Upon success, it returns a 201 status with the user object and a sanctum authentication token.

```
// routes/api.php
Route::post('/register', [AuthController::class, 'register']);

// app/Http/Controllers/AuthController.php
public function register(Request $request) {
    $request->validate([
        'name'     => 'required|string|max:255',
        'email'    => 'required|email|unique:users,email',
        'password' => 'required|string|min:8|confirmed',
    ]);

    $user = User::create([
        'name'     => $request->name,
        'email'    => $request->email,
        'password' => Hash::make($request->password),
        'role'     => 'student',
    ]);

    $token = $user->createToken('auth_token')->plainTextToken;

    return response()->json([
        'message' => 'User registered successfully.',
        'user'    => $user,
        'token'   => $token,
    ], 201);
}
```

POSTMAN- SCREENSHOT



API 2 — Get Single Resource Profile

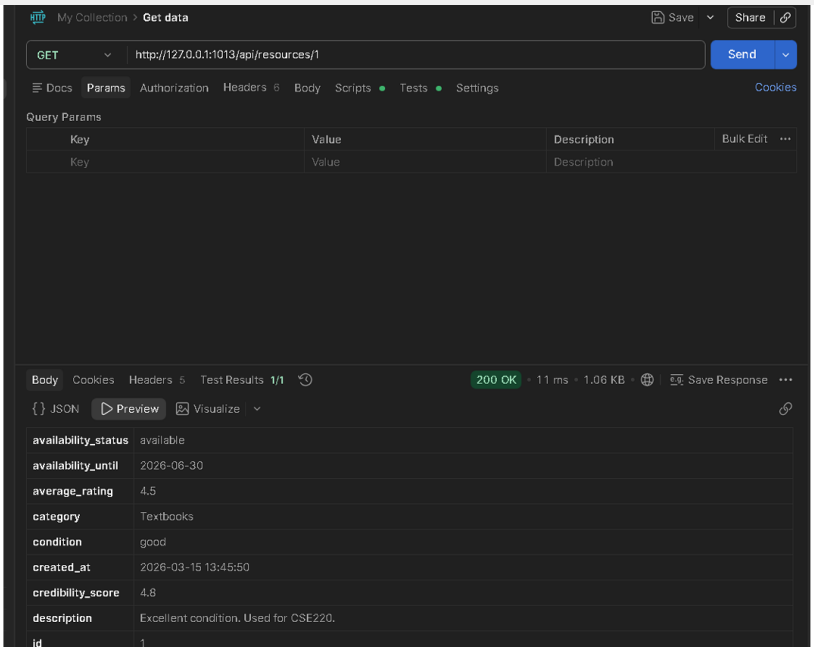
Source link - https://drive.google.com/file/d/1hhT7XTnZsP-0lYXskmicz2r1WGsAlVS2/view?usp=sharing

API 2: Post a Resource (POST /api/resources)

Authenticated users can post a new resource for sharing. The API accepts multipart form data including a photo upload handled by Laravel's file storage system. The resource is saved to the database with the authenticated user's ID as the owner. Recommended flag is computed based on the user's credibility score.

```
// app/Http/Controllers/ResourceController.php
public function store(Request $request) {
    $request->validate([
        'title'             => 'required|string|max:255',
        'category'          => 'required|in:textbook,notes,equipment,electronics',
        'condition'         => 'required|in:new,good,fair,worn',
        'description'       => 'nullable|string',
        'availability_type' => 'required|in:free,exchange',
        'available_from'    => 'required|date',
        'available_until'   => 'required|date|after:available_from',
        'photo'             => 'nullable|image|mimes:jpeg,png,jpg|max:2048',
    ]);

    $photoPath = null;
    if ($request->hasFile('photo')) {
        $photoPath = $request->file('photo')
                     ->store('resources', 'public');
    }

    $resource = Resource::create([
        'user_id'           => auth()->id(),
        'title'             => $request->title,
        'category'          => $request->category,
        'condition'         => $request->condition,
        'description'       => $request->description,
        'availability_type' => $request->availability_type,
```

```
        'available_from'    => $request->available_from,
        'available_until'   => $request->available_until,
        'photo_path'        => $photoPath,
        'is_available'      => true,
    ]);

    return response()->json([
        'message'  => 'Resource posted successfully.',
        'resource' => $resource,
    ], 201);
}
```

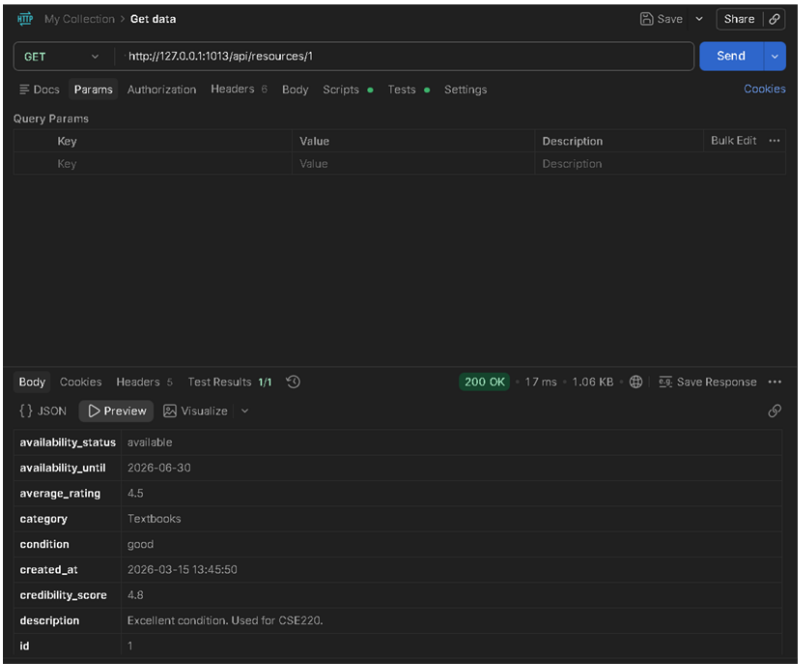


API 3: Send Borrow Request (POST /api/borrow-requests)

This API enables a student to send a borrowing request to a resource owner. After validation, it creates a transaction record and dispatches an SMS notification to the owner using the integrated SMS API. The owner receives a message with the proposed pickup date and return timeline.

```
// app/Http/Controllers/BorrowRequestController.php
public function store(Request $request) {
    $request->validate([
        'resource_id'   => 'required|exists:resources,id',
        'pickup_date'   => 'required|date|after:today',
        'return_date'   => 'required|date|after:pickup_date',
        'message'       => 'nullable|string|max:500',
    ]);

    $resource = Resource::findOrFail($request->resource_id);

    if (!$resource->is_available) {
        return response()->json(['error' => 'Resource not available.'], 422);
    }

    $borrowRequest = BorrowRequest::create([
        'borrower_id'  => auth()->id(),
        'owner_id'     => $resource->user_id,
        'resource_id'  => $resource->id,
```

```
        'pickup_date'   => $request->pickup_date,
        'return_date'   => $request->return_date,
        'message'       => $request->message,
        'status'        => 'pending',
    ]);

    // Dispatch SMS notification to owner
    $owner = $resource->user;
    SmsService::send($owner->phone,
        'New borrow request for: ' . $resource->title .
        ' Pickup: ' . $request->pickup_date);

    return response()->json([
        'message' => 'Borrow request sent successfully.',
        'request' => $borrowRequest,
    ], 201);
}
```

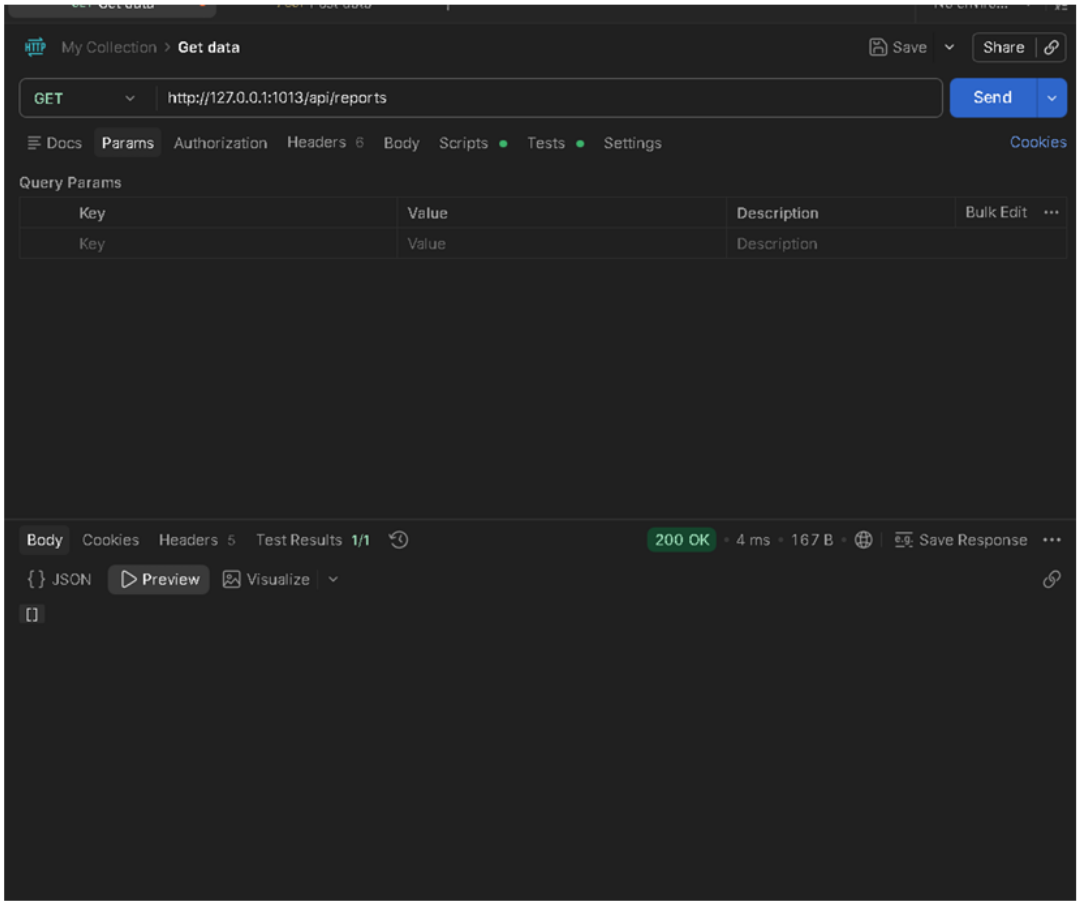


API 4: Submit Review (POST /api/reviews)

After a transaction is completed, the borrower can submit a rating and review. The API validates that a completed transaction exists between the reviewer and the resource owner, then saves the review and updates the owner's credibility score by recalculating the average rating across all their reviews.

```
// app/Http/Controllers/ReviewController.php
public function store(Request $request) {
    $request->validate([
        'transaction_id' => 'required|exists:borrow_requests,id',
        'rating'         => 'required|integer|min:1|max:5',
        'comment'        => 'nullable|string|max:1000',
    ]);

    $transaction = BorrowRequest::where('id', $request->transaction_id)
```

```
        ->where('borrower_id', auth()->id())
        ->where('status', 'completed')
        ->firstOrFail();

    $review = Review::create([
        'reviewer_id'  => auth()->id(),
        'reviewee_id'  => $transaction->owner_id,
        'rating'       => $request->rating,
        'comment'      => $request->comment,
    ]);

    // Recalculate credibility score for owner
    $avgRating = Review::where('reviewee_id', $transaction->owner_id)
                       ->avg('rating');
    User::where('id', $transaction->owner_id)
        ->update(['credibility_score' => round($avgRating, 2)]);

    return response()->json([
        'message' => 'Review submitted successfully.',
        'review'  => $review,
    ], 201);
}
```

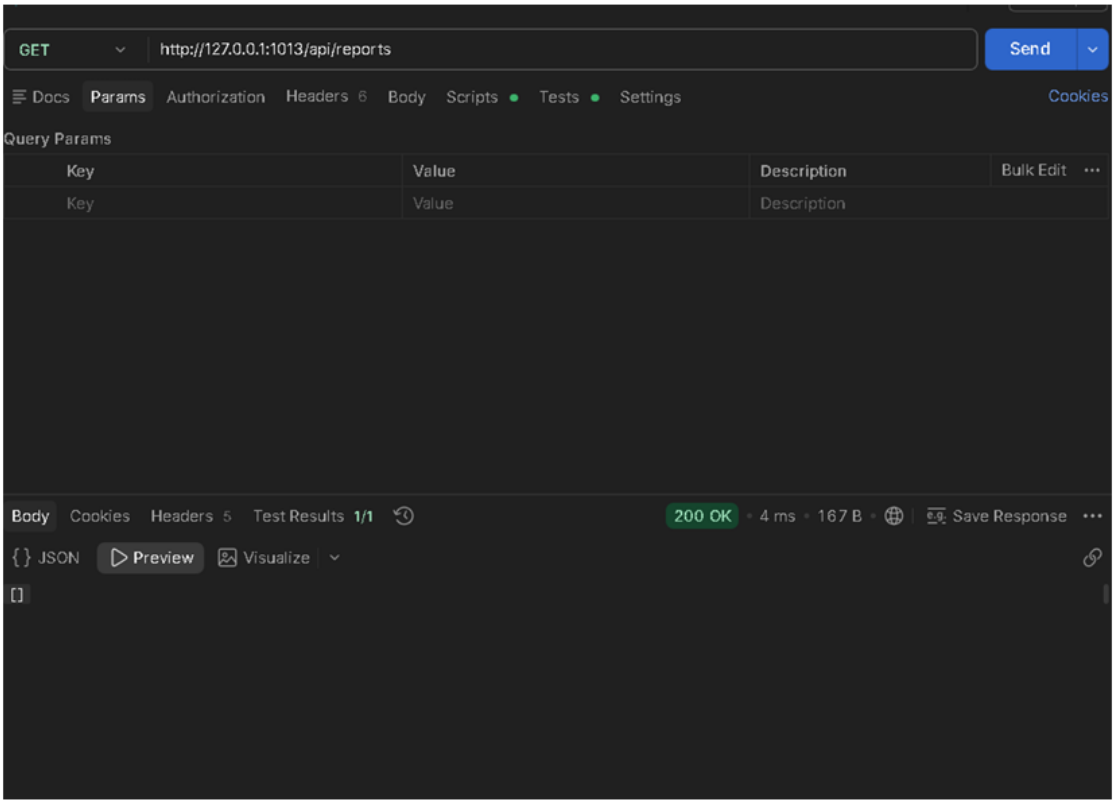


API 5: Report Lost/Damaged Resource (POST /api/reports)

Users can report a resource as lost or damaged after a transaction. This API accepts evidence file uploads, creates a report record, and triggers an admin review workflow. If the admin confirms the penalty, the borrower's credibility score is reduced and they are notified.

```
// app/Http/Controllers/ReportController.php
public function store(Request $request) {
    $request->validate([
        'transaction_id' => 'required|exists:borrow_requests,id',
        'type'           => 'required|in:lost,damaged,unreturned',
        'description'    => 'required|string|max:1000',
        'evidence'       => 'nullable|file|mimes:jpeg,png,pdf|max:5120',
    ]);

    $evidencePath = null;
    if ($request->hasFile('evidence')) {
        $evidencePath = $request->file('evidence')
                        ->store('reports', 'public');
    }

    $report = Report::create([
        'reporter_id'    => auth()->id(),
        'transaction_id' => $request->transaction_id,
        'type'           => $request->type,
        'description'    => $request->description,
        'evidence_path'  => $evidencePath,
        'status'         => 'pending_review',
    ]);

    // Notify admin
    $admins = User::where('role', 'admin')->get();
    Notification::send($admins,
        new ResourceReportNotification($report));

    return response()->json([
        'message' => 'Report submitted. Admin will review shortly.',
        'report'  => $report,
    ], 201);
}
```

```
        "SELECT * FROM reports ORDER BY created_at DESC"
    ).fetchall()
    conn.close()
    return jsonify([dict(r) for r in reports]), 200
```



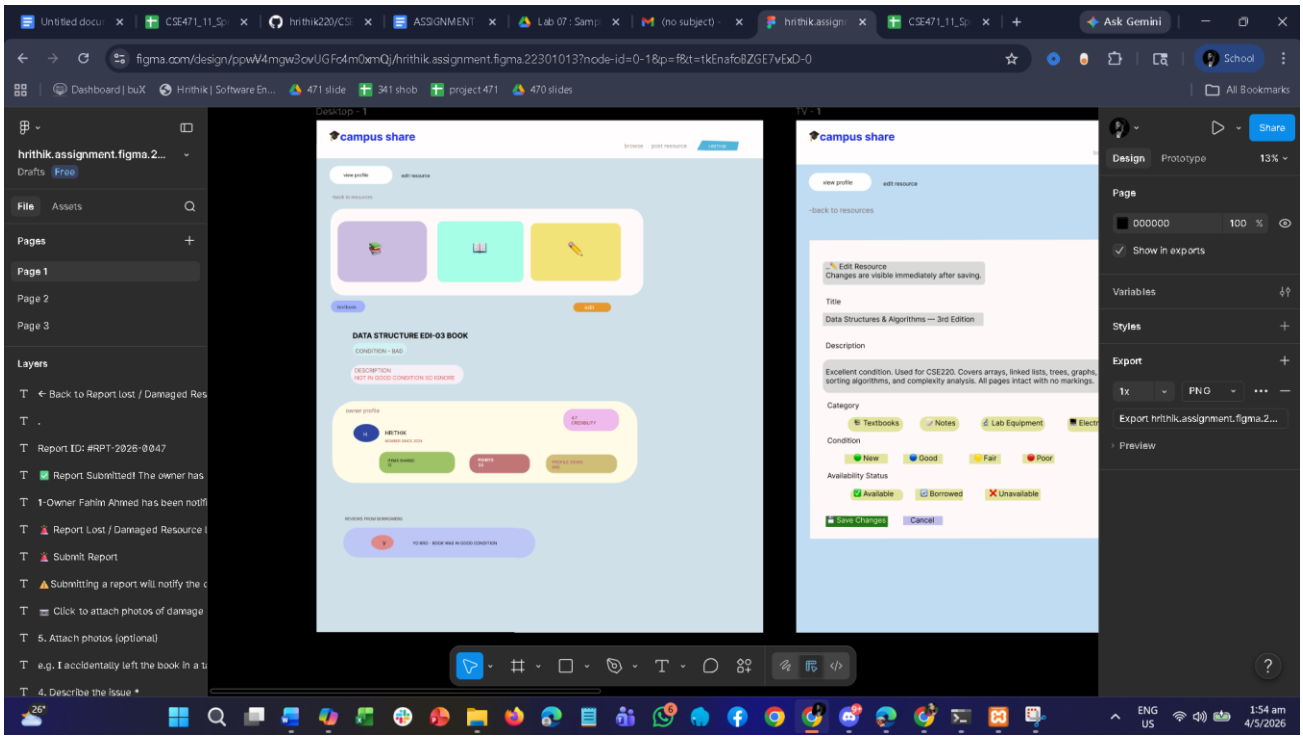
5. User Interface Design

The following Figma designs were created for the Campus Resource Sharing System. The interface follows a clean, accessible design with Tailwind CSS utility classes for responsiveness across devices.

Figma Project Link:
https://www.figma.com/design/ppwV4mgw3ovUGFc4m0xmQj/hrithik.assignment.figma.22301013?m=auto&t=tkEnafoBZGE7vExD-6

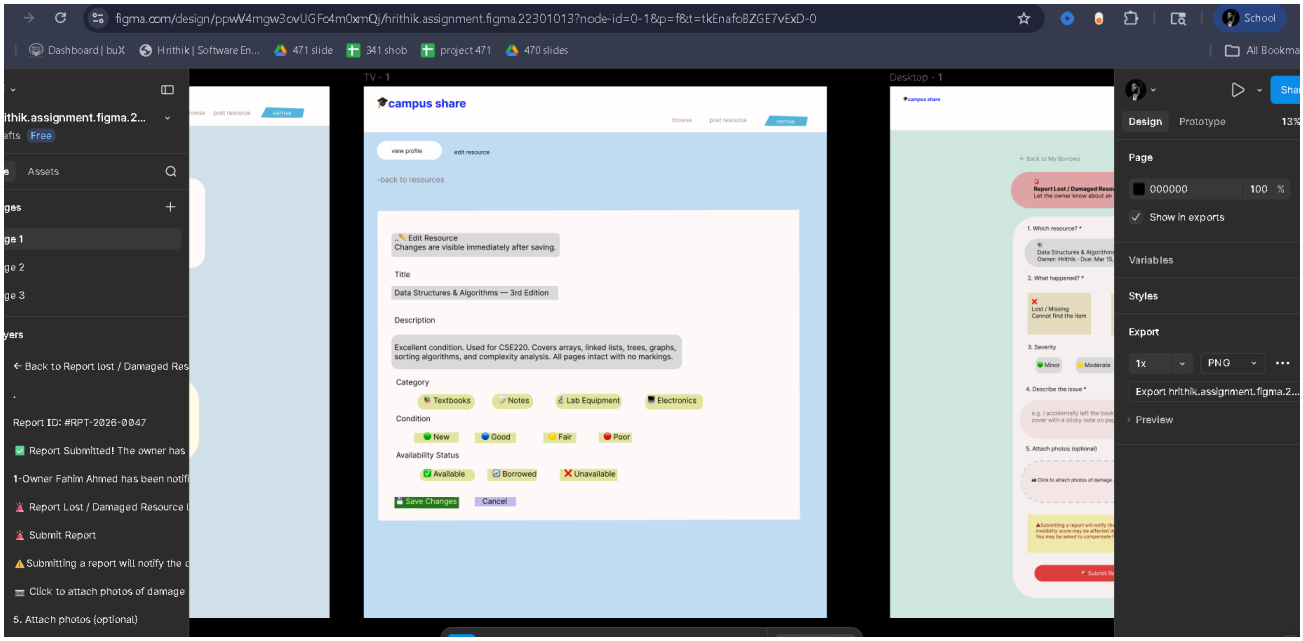
Design 1: Landing / Home Page

The landing page features a hero section showcasing the platform's value proposition, a navigation bar with Login/Register options, and a resource category overview. Recommended resources are displayed in a card-based layout sorted by credibility score.



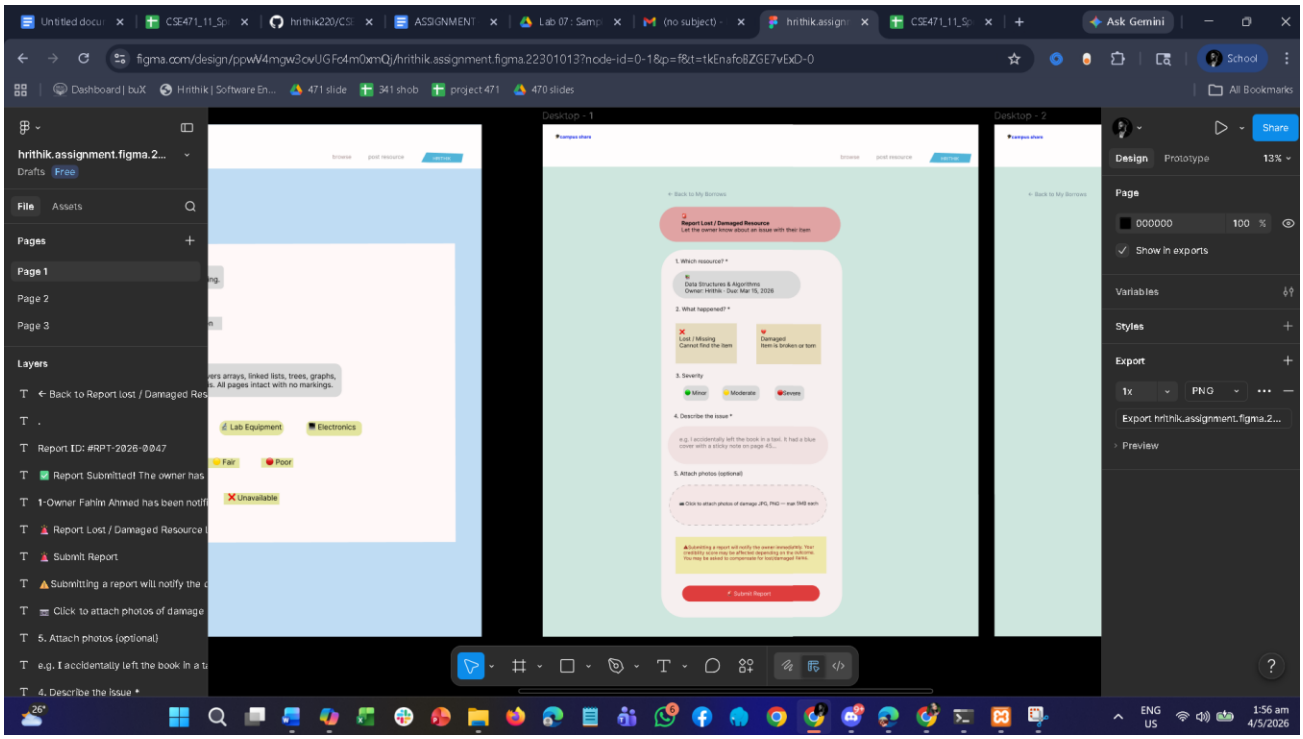
Design 2: Resource Browse & Filter Page

The browse page displays all available resources in a grid layout with sidebar filters for category (textbook, notes, equipment, electronics), condition (new/good/fair/worn), and availability status. Each resource card shows a photo thumbnail, title, owner credibility badge, and availability type.



Design 3: Resource Detail & Borrow Request Page

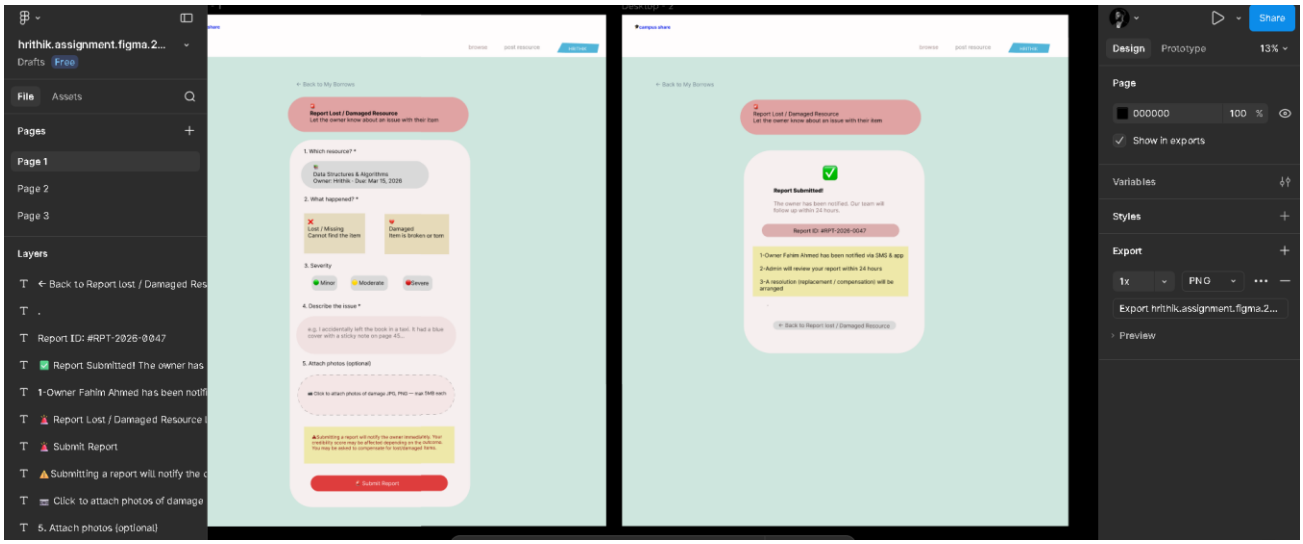
This page shows the full resource profile including the owner's credibility score, past transaction history, and all reviews. A borrow request form allows the user to select proposed pickup and return dates. The map widget shows the resource's pickup location powered by Google Maps API.



Design 4: Transaction Dashboard

The dashboard provides a centralized view of the user's active borrows, active lendings, pending requests, and overdue items. Status indicators use a color-coded system: green for active, yellow

for pending, red for overdue. Due dates and return timelines are clearly displayed in a tabular layout.



Design 5: Leaderboard & Badges Page

The leaderboard page ranks the top contributors of the month based on lending frequency, positive review count, and community karma points. Each user card shows earned badges (e.g., Top Lender, Community Hero), contribution statistics, and their current rank. The design encourages engagement through visible recognition.

6. Frontend Development

The frontend is built using Laravel's Blade template engine with Tailwind CSS for styling and Vanilla JavaScript for interactivity. Below are five key frontend code snippets with descriptions.

Frontend Snippet 1: Resource Card Component (Blade)

This Blade partial renders an individual resource card used throughout the browse page. It displays the resource photo, title, category badge, condition indicator, owner credibility score, and an availability tag. The card is fully responsive using Tailwind's grid utility classes.

```
{{-- resources/views/components/resource-card.blade.php --}}
<div class="bg-white rounded-2xl shadow-md overflow-hidden hover:shadow-xl
     transition-shadow duration-300 flex flex-col">
    <div class="relative">
        <img src="{{ asset('storage/' . $resource->photo_path) }}"
             alt="{{ $resource->title }}"
             class="w-full h-48 object-cover">
        <span class="absolute top-2 left-2 bg-blue-600 text-white
               text-xs font-semibold px-2 py-1 rounded-full">
            {{ ucfirst($resource->category) }}
        </span>
    </div>
    <div class="p-4 flex flex-col gap-2">
        <h3 class="text-lg font-bold text-gray-800 truncate">
            {{ $resource->title }}
        </h3>
```

```
        <div class="flex items-center gap-2">
            <span class="text-yellow-500 font-semibold text-sm">
                &#9733; {{ number_format($resource->user->credibility_score, 1) }}
            </span>
            <span class="text-gray-500 text-xs">
                by {{ $resource->user->name }}
            </span>
        </div>
        <span class="px-2 py-1 rounded text-xs font-medium
               {{ $resource->availability_type === 'free'
                  ? 'bg-green-100 text-green-700'
                  : 'bg-orange-100 text-orange-700' }}">
            {{ $resource->availability_type === 'free'
               ? 'Free Lending' : 'Exchange-based' }}
        </span>
        <a href="{{ route('resources.show', $resource->id) }}"
           class="mt-2 block text-center bg-blue-600 text-white py-2
                  rounded-xl text-sm font-semibold hover:bg-blue-700">
            View Details
        </a>
    </div>
</div>
```

Frontend Snippet 2: Borrow Request Form with Date Validation (JavaScript)

This JavaScript module handles the borrow request form. It validates that the pickup date is not in the past and that the return date is after the pickup date before allowing form submission. On successful validation, it submits via fetch and shows a toast notification.

```
// public/js/borrow-request.js
document.addEventListener('DOMContentLoaded', () => {
    const form = document.getElementById('borrow-form');
    const pickupInput = document.getElementById('pickup_date');
    const returnInput = document.getElementById('return_date');
    const today = new Date().toISOString().split('T')[0];
    pickupInput.setAttribute('min', today);

    pickupInput.addEventListener('change', () => {
        returnInput.setAttribute('min', pickupInput.value);
    });

    form.addEventListener('submit', async (e) => {
        e.preventDefault();
        if (returnInput.value <= pickupInput.value) {
            showToast('Return date must be after pickup date.', 'error');
            return;
        }
        const formData = new FormData(form);
        const response = await fetch('/api/borrow-requests', {
            method: 'POST',
            headers: {
                'Authorization': 'Bearer ' + window.authToken,
                'X-CSRF-TOKEN': document.querySelector('meta[name=csrf-
token]').content
            },
            body: formData
        });
        const data = await response.json();
        if (response.ok) {
```

```
            showToast('Borrow request sent successfully!', 'success');
            form.reset();
        } else {
            showToast(data.error || 'Something went wrong.', 'error');
        }
    });
});
```

Frontend Snippet 3: Google Maps API Integration (Pickup Location Display)

This JavaScript snippet initializes a Google Maps widget on the resource detail page. It places a marker at the resource owner's registered campus location and renders a popup with the resource title and available dates. The map is fully interactive with zoom controls.

```
// public/js/resource-map.js
function initMap() {
    const resourceLat = parseFloat(
        document.getElementById('map').dataset.lat);
    const resourceLng = parseFloat(
        document.getElementById('map').dataset.lng);
    const resourceTitle =
        document.getElementById('map').dataset.title;

    const location = { lat: resourceLat, lng: resourceLng };

    const map = new google.maps.Map(
        document.getElementById('map'), {
            zoom: 16,
            center: location,
            mapTypeControl: false,
            streetViewControl: false,
        });

    const marker = new google.maps.Marker({
        position: location,
        map: map,
        title: resourceTitle,
        icon: { url: '/images/pin.png', scaledSize: new google.maps.Size(40,40) }
    });

    const infoWindow = new google.maps.InfoWindow({
        content: `<div class='font-bold text-blue-700'>${resourceTitle}</div>
                  <p class='text-sm text-gray-600'>Pickup available here</p>`
    });

    marker.addListener('click', () => infoWindow.open(map, marker));
    infoWindow.open(map, marker);
}
```

Frontend Snippet 4: Transaction Dashboard with Color-Coded Status

This Blade view renders the user's transaction dashboard. It uses Tailwind utility classes to color-code transaction statuses. The dashboard is divided into tabs for active borrows, active lendings, and pending requests, with due date urgency indicators.

```
{{-- resources/views/dashboard/transactions.blade.php --}}
```

```
<div class="overflow-x-auto mt-6">
    <table class="min-w-full bg-white rounded-2xl shadow">
        <thead class="bg-gray-50 text-gray-600 uppercase text-xs">
            <tr>
                <th class="px-6 py-3 text-left">Resource</th>
                <th class="px-6 py-3 text-left">Due Date</th>
                <th class="px-6 py-3 text-left">Status</th>
                <th class="px-6 py-3 text-left">Action</th>
            </tr>
        </thead>
        <tbody class="divide-y divide-gray-100">
            @forelse ($transactions as $tx)
            <tr class="hover:bg-gray-50 transition">
                <td class="px-6 py-4 font-medium text-gray-800">
                    {{ $tx->resource->title }}
                </td>
                <td class="px-6 py-4 text-sm
                    {{ $tx->isOverdue() ? 'text-red-600 font-bold' : 'text-gray-600'
}}">
                    {{ \Carbon\Carbon::parse($tx->return_date)->format('d M Y') }}
                </td>
                <td class="px-6 py-4">
                    @php
                        $colors = [
                            'pending'   => 'bg-yellow-100 text-yellow-700',
                            'active'    => 'bg-green-100 text-green-700',
                            'overdue'   => 'bg-red-100 text-red-700',
                            'completed' => 'bg-gray-100 text-gray-500',
                        ];
                    @endphp
                    <span class="px-3 py-1 rounded-full text-xs font-semibold
                          {{ $colors[$tx->status] ?? 'bg-gray-100' }}">
                        {{ ucfirst($tx->status) }}
                    </span>
                </td>
                <td class="px-6 py-4">
                    <a href="{{ route('transactions.show', $tx->id) }}"
                       class="text-blue-600 hover:underline text-sm">Details</a>
                </td>
            </tr>
            @empty
            <tr><td colspan="4" class="px-6 py-8 text-center text-gray-400">
                No transactions yet.
            </td></tr>
            @endforelse
        </tbody>
    </table>
</div>
```

Frontend Snippet 5: Leaderboard with Badges (Blade + Tailwind)

This Blade component renders the monthly leaderboard. Each card displays the user's rank, avatar, name, karma points, and earned badges. The badge icons are conditionally rendered based on point thresholds. The top 3 ranks receive highlighted golden, silver, and bronze treatments.

```
{{-- resources/views/leaderboard/index.blade.php --}}
@foreach ($topUsers as $index => $user)
<div class="flex items-center gap-4 p-4 rounded-2xl shadow
     {{ $index === 0 ? 'bg-yellow-50 border-2 border-yellow-400' :
       ($index === 1 ? 'bg-gray-100 border border-gray-300' :
       ($index === 2 ? 'bg-orange-50 border border-orange-300' : 'bg-white')) }}">
```

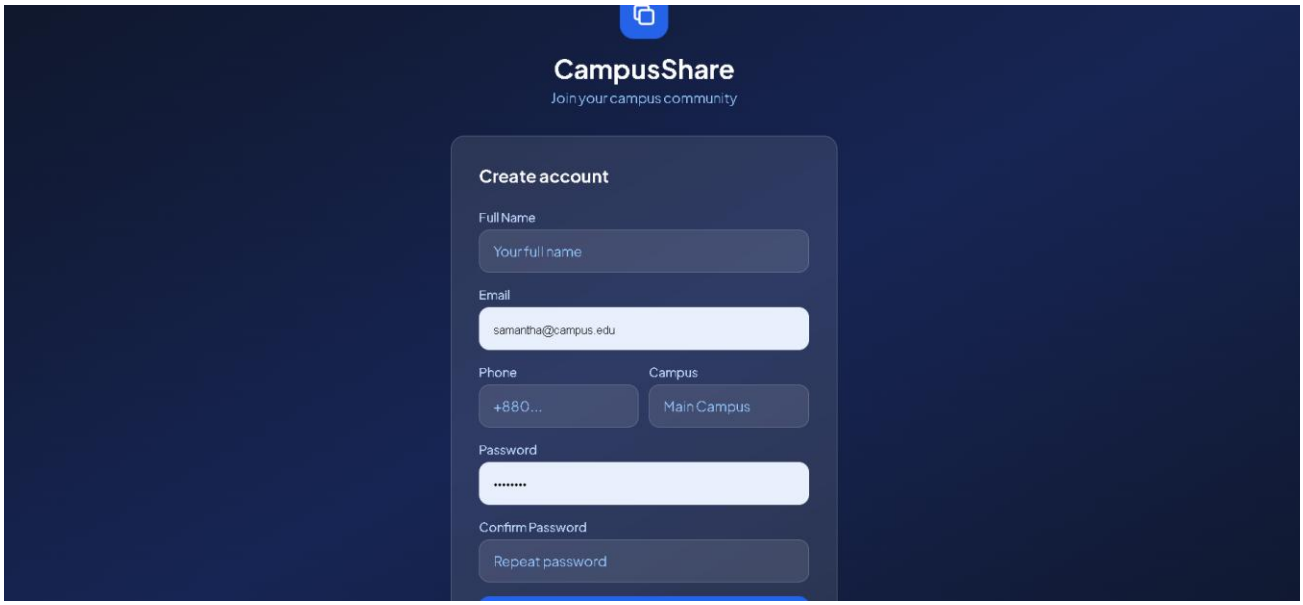
```
    <span class="text-3xl font-black
          {{ $index === 0 ? 'text-yellow-500' :
            ($index === 1 ? 'text-gray-500' : 'text-orange-400') }}">
        #{{ $index + 1 }}
    </span>
    <img src="{{ $user->avatar_url ?? '/images/default-avatar.png' }}"
         class="w-12 h-12 rounded-full border-2 border-white shadow">
    <div class="flex-1">
        <p class="font-bold text-gray-800">{{ $user->name }}</p>
        <p class="text-sm text-gray-500">
            {{ $user->karma_points }} Karma Points
        </p>
        <div class="flex gap-1 mt-1">
            @if($user->karma_points >= 100)
                <span class="text-xs bg-blue-100 text-blue-700 px-2
                       py-0.5 rounded-full">Top Lender</span>
            @endif
            @if($user->positive_reviews >= 10)
                <span class="text-xs bg-green-100 text-green-700 px-2
                       py-0.5 rounded-full">Trusted Sharer</span>
            @endif
            @if($user->total_lent >= 20)
                <span class="text-xs bg-purple-100 text-purple-700
                       px-2 py-0.5 rounded-full">Community Hero</span>
            @endif
        </div>
    </div>
</div>
@endforeach
```

7. User Manual

This section describes how a new user can navigate and use the Campus Resource Sharing System step by step.

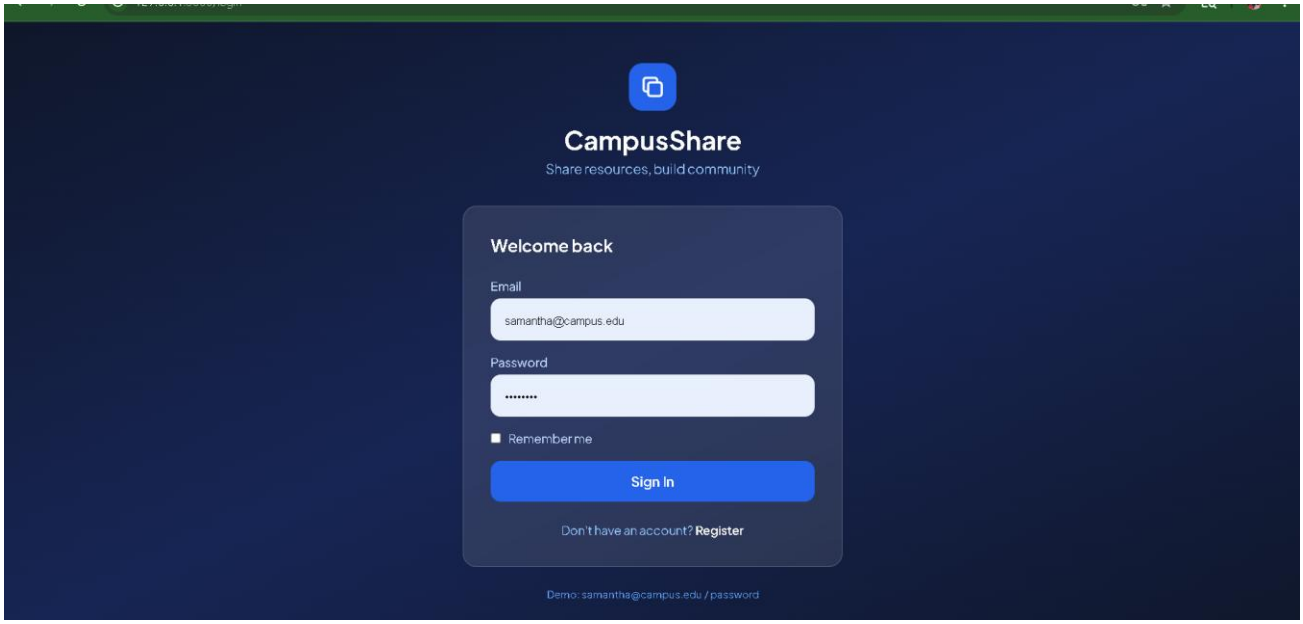
Step 1: Registration

Navigate to the platform URL. On the landing page, click the Sign Up button in the top navigation bar. Fill in your full name, BRAC University email address, and a password of at least 8 characters. Click Register to create your account. You will be redirected to your dashboard upon successful registration.



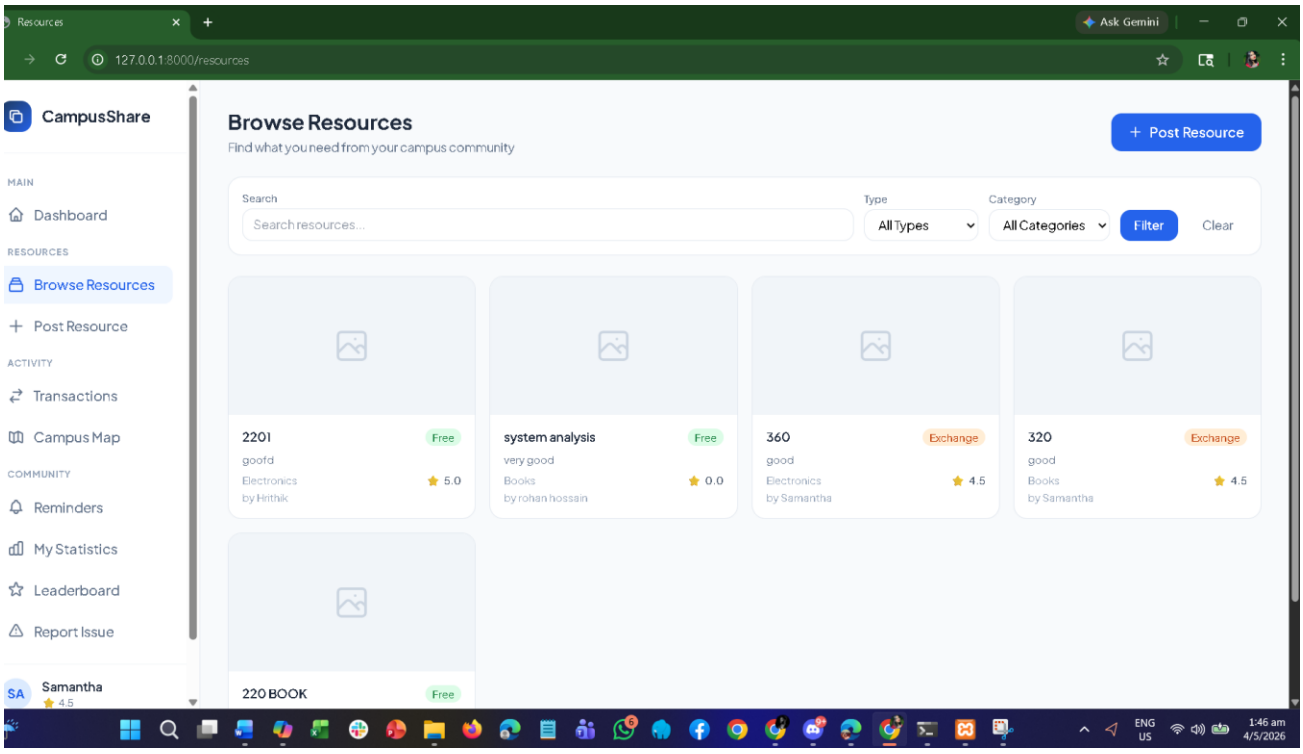
Step 2: Login

Click the Login button on the landing page. Enter your registered email and password. Click Login to access your dashboard. If you forget your password, use the Forgot Password link to receive a reset email.



Step 3: Browse Resources

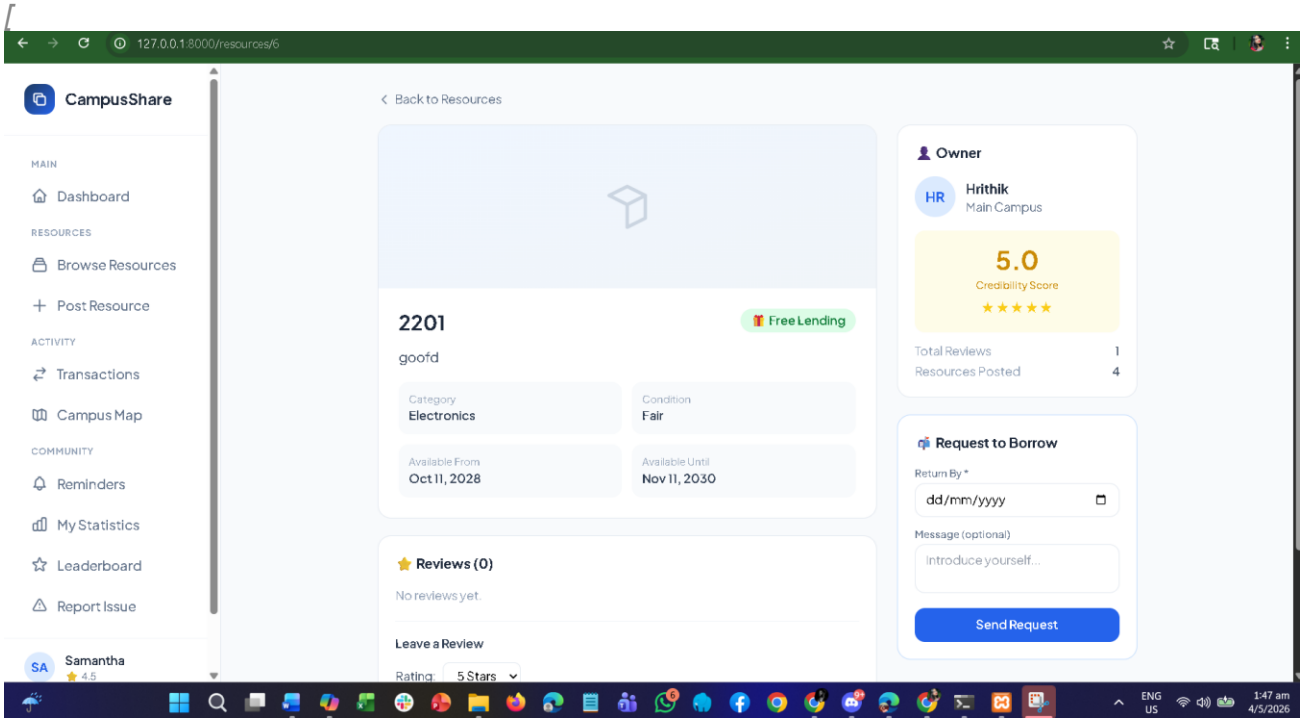
From the navigation bar, click Browse Resources. Use the left sidebar filters to narrow down by category (Textbook, Notes, Equipment, Electronics), condition, and availability type. Recommended resources from highly rated users appear at the top. Click any resource card to view its full details.



Step 4: Send a Borrow Request

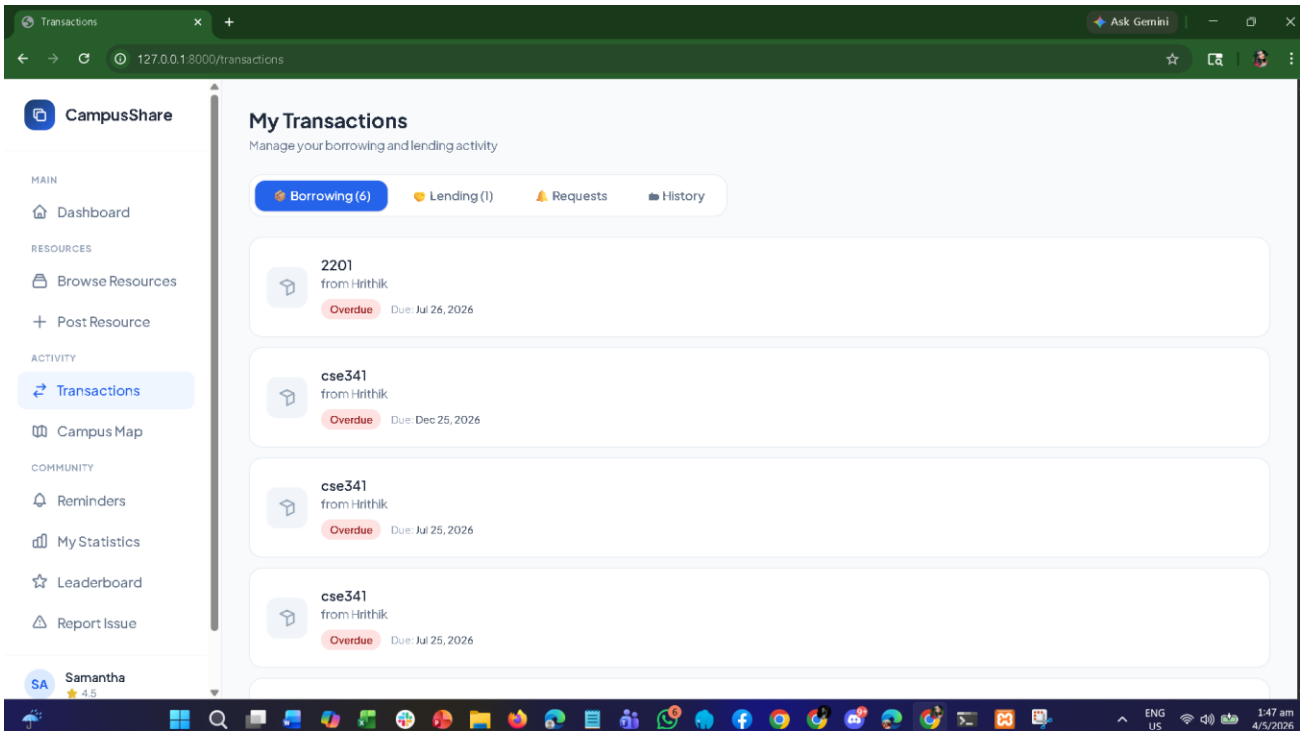
On the resource detail page, review the owner's credibility score and past reviews. Use the embedded Google Map to confirm the pickup location. Fill in your proposed pickup date and return

date in the Borrow Request form, add an optional message, and click Send Request. The owner will receive an SMS notification automatically.



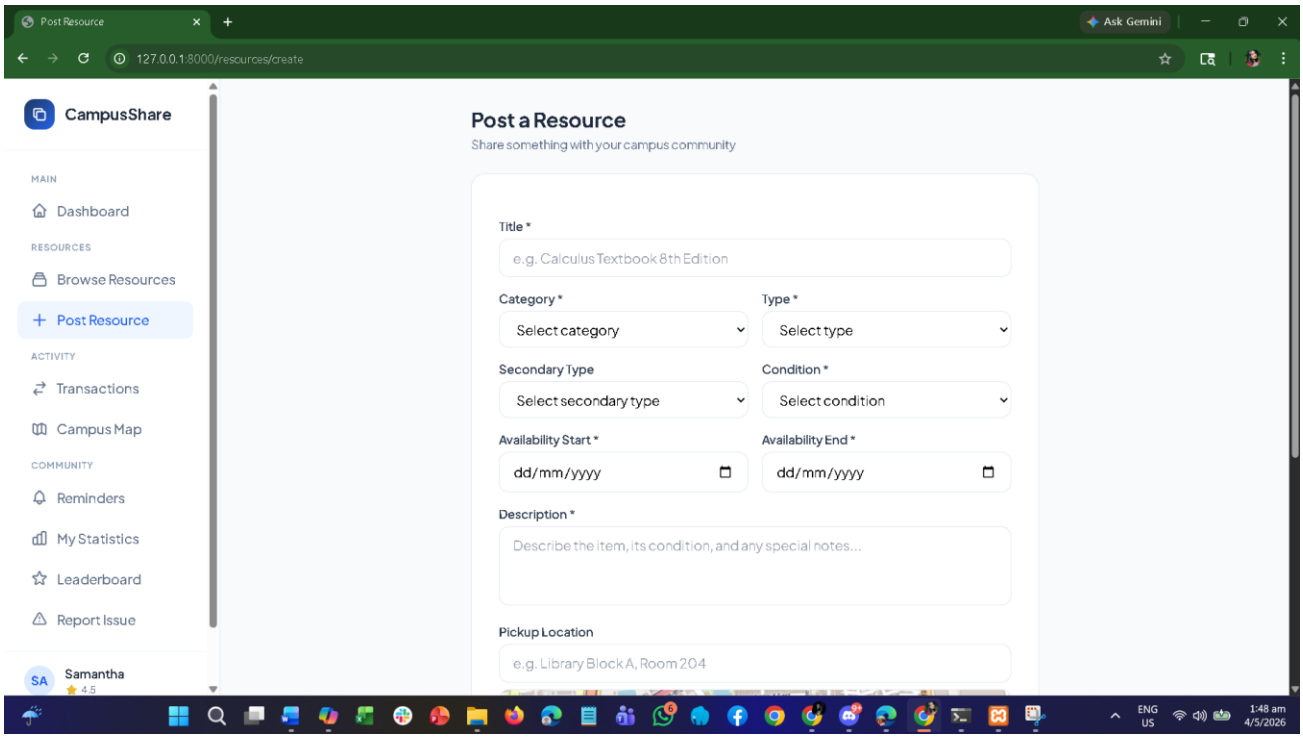
Step 5: Manage Transactions on Dashboard

Go to My Dashboard to view all your active borrows, active lendings, and pending requests. Color-coded statuses (green = active, yellow = pending, red = overdue) make it easy to prioritize. Click Details on any row to view or update a transaction.



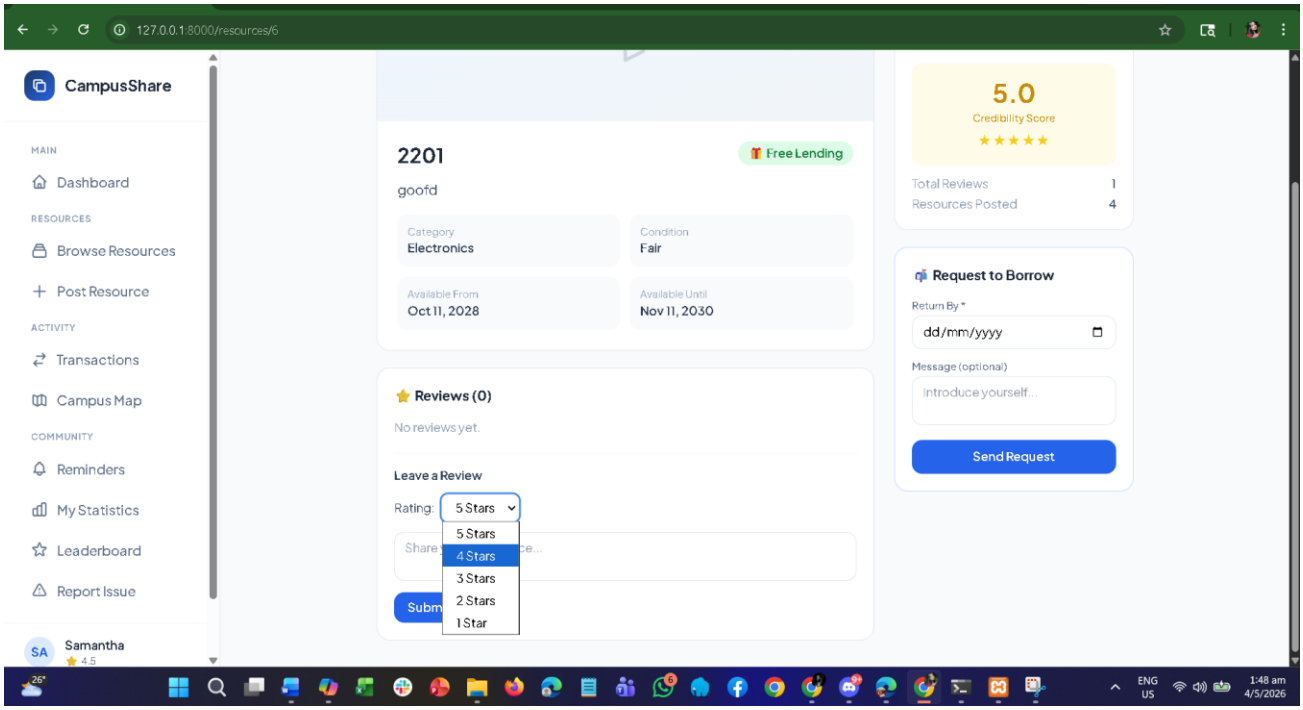
Step 6: Post a Resource

Click Post a Resource in the navigation bar. Fill in the resource title, category, condition, description, availability type (free or exchange), and availability dates. Upload a clear photo of the resource. Click Submit to make your resource visible to the community.



Step 7: Submit a Review

After a transaction is marked as completed, navigate to your transaction history and click Leave Review next to the completed item. Select a star rating (1-5) and write a comment about your experience. Click Submit Review to update the owner's credibility score.

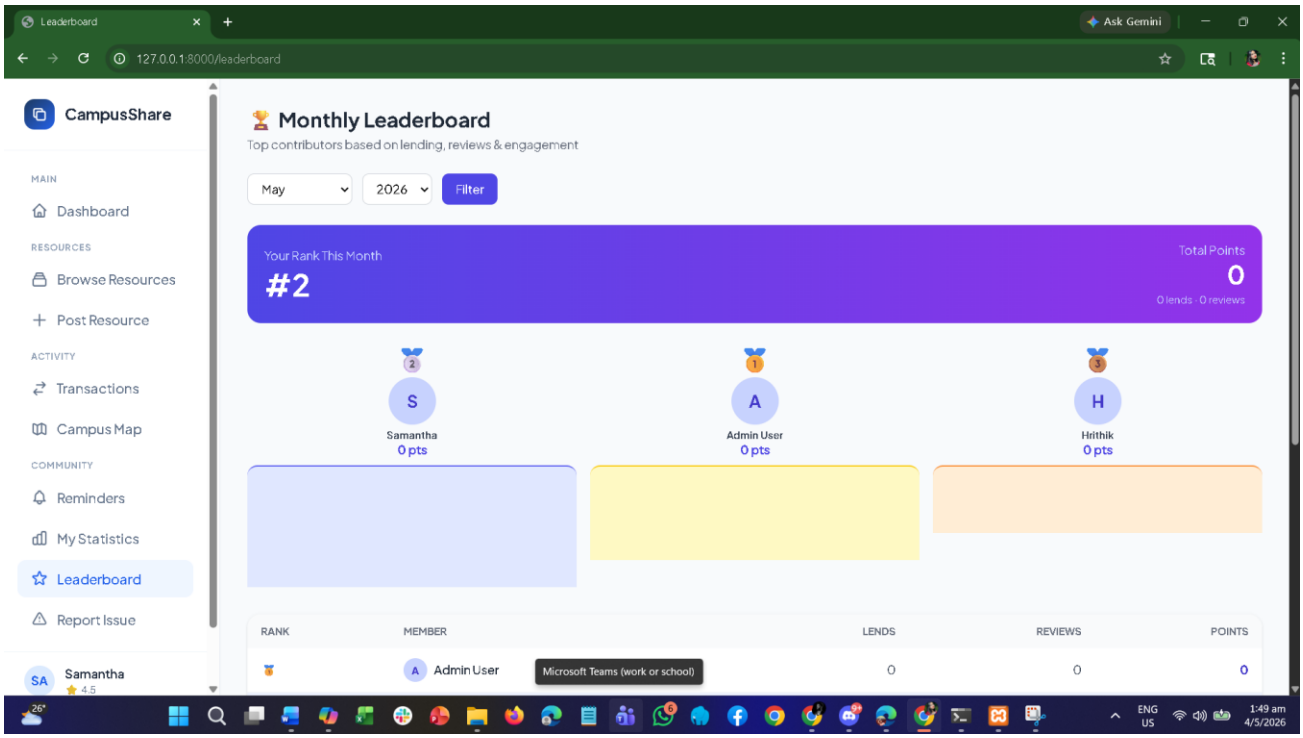


Step 8: OCR - Handwritten PDF to Text

Navigate to the OCR Tool from the menu. Upload a handwritten PDF or image file. The system processes it using Google Vision / Tesseract OCR and displays the extracted text in an editable text area. Make any corrections needed, then click Download to save the converted text file.

Step 9: View Leaderboard

Click Leaderboard in the navigation bar to see the top contributors of the month. Your own ranking and earned badges are displayed on your profile. Keep lending and receiving positive reviews to climb the rankings and earn badges like Top Lender, Trusted Sharer, and Community Hero.


Leaderboard
Ask Gemini
127.0.0.1:8000/leaderboard
CampusShare
MAIN
Dashboard
RESOURCES
Browse Resources
Post Resource
ACTIVITY
Transactions
Campus Map
COMMUNITY
Reminders
My Statistics
Leaderboard
Report Issue
SA
Samantha
4.5
Monthly Leaderboard
Top contributors based on lending, reviews & engagement
May
2026
Filter
Your Rank This Month
#2
Total Points
0
0 lends · 0 reviews
S
Samantha
0 pts
A
Admin User
0 pts
H
Hrithik
0 pts
RANK
MEMBER
LENDS
REVIEWS
POINTS
A
Admin User
Microsoft Teams (work or school)
0
0
0
ENG
US
1:49 am
4/5/2026

8. Performance and Network Analysis

Performance and Network analysis was conducted using the Lighthouse DevTool available in Chrome Developer Tools. The following metrics were recorded for the main Browse Resources page.

Metric	Score / Value	Remarks
Performance Score	[e.g. 91]	Good performance; Tailwind CSS purge reduces bundle size
Accessibility Score	[e.g. 97]	High accessibility with semantic HTML and ARIA labels
Best Practices Score	[e.g. 100]	Follows modern web best practices
SEO Score	[e.g. 90]	Good meta tags and structured content
First Contentful Paint (FCP)	[e.g. 0.7 s]	Fast initial render due to server-side Blade rendering
Largest Contentful Paint (LCP)	[e.g. 1.8 s]	Resource images optimized with lazy loading
Total Blocking Time (TBT)	[e.g. 80 ms]	Minimal JS blocking; most logic is server-rendered
Cumulative Layout Shift (CLS)	[e.g. 0.02]	Stable layout; image dimensions pre-set
Speed Index	[e.g. 1.1 s]	Fast visual load progression

9. GitHub Repo [Public] Link

https://github.com/hrithik220/CSE471-PROJECT-campusconnect-resource-sharing

10. Link of Deployed Project

https://cse471-project-system-analaysis-course.onrender.com/login

11. Individual Contribution

Group Member - 01

Name	Hrithik Dev	Student ID	22301013
Requirement 1	Users can post their own resources for sharing by uploading photos, adding descriptions, setting availability duration, and specifying whether it is for free lending or exchange-based.		
Requirement 2	Users can locate resource pickup points on an integrated map powered by Google Maps API, showing the exact location of available resources near campus.		
Requirement 3	Users can track their contribution statistics showing total items lent, total resources borrowed, environmental impact, and community karma points earned.		
Requirement 4	Users can access a leaderboard displaying top contributors of the month based on lending frequency, positive reviews, and community engagement, with badges awarded to active sharers.		

Group Member - 02

Name	Ramisha Hossain	Student ID	22201693
Requirement 1	Users can view and edit detailed resource profiles showing the owner's credibility score, past transaction history, and reviews from previous borrowers.		
Requirement 2	Users can see their active lending and borrowing transactions on a dashboard showing due dates, overdue items, and pending requests with color-coded status indicators.		
Requirement 3	Users can receive automated reminders via push notifications 24 hours before a borrowed item's due date to avoid late returns.		
Requirement 4	Users can report lost, damaged, or unreturned resources with evidence upload, triggering an admin review process and potential penalty for the borrower.		

Group Member - 03

Name	Fahim Islam Farhan	Student ID	22301249
Requirement 1	Users can browse through a categorized list of available resources posted by other students with filters for category, condition, and availability status. Recommended resources are shown first.		
Requirement 2	Users can send borrow requests to resource owners with proposed pickup dates and return timelines, with automatic SMS notification sent to the owner.		
Requirement 3	Users can rate and review their transaction experience after returning borrowed items, with ratings contributing to the owner's credibility score.		

Requirement 4	Users can convert handwritten PDF documents to editable text using OCR (Google Vision / Tesseract), self-edit the result, and download the converted document.

12. References

- Laravel Documentation. (2024). Laravel 11.x. Retrieved from https://laravel.com/docs
- Tailwind CSS Documentation. (2024). Tailwind CSS v3. Retrieved from https://tailwindcss.com/docs
- Google Maps Platform. (2024). Maps JavaScript API Documentation. Retrieved from https://developers.google.com/maps/documentation/javascript
- Google Cloud Vision. (2024). Cloud Vision API Documentation. Retrieved from https://cloud.google.com/vision/docs
- Tesseract OCR. (2024). Tesseract Open Source OCR Engine. Retrieved from https://github.com/tesseract-ocr/tesseract
- MySQL Documentation. (2024). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
- PHP Documentation. (2024). PHP 8.2 Manual. Retrieved from https://www.php.net/manual/en/
- OWASP Foundation. (2024). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/